

Department of Mathematics and Computer Science

Master's thesis

Interactive Visualization Framework for DNA Data Storage using De Bruijn Graphs

Florian Schwarz

July 19th, 2023

Dr. Georges Hattab
Prof. Dr. Dominik Heider

Thesis Statement

I herewith formally declare that I, Florian Schwarz, have written the submitted thesis independently. I did not use any outside support except for the quoted literature and other sources mentioned in the paper. I clearly marked and separately listed all of the literature and all of the other sources which I employed when producing this academic work, either literally or in content. This thesis has not been handed in or published before in the same or similar form.

I am aware, that in case of an attempt at deception based on plagiarism, the thesis would be graded under 5 points and counted as one failed examination attempt. The thesis may only be repeated once.

In the submitted thesis the written copies and the electronic version for archiving are identical in content.

Ort/Place, Datum/Date

Unterschrift des Autors/Signature of author

Acknowledgement

First of all, I want to thank my supervisors Chisom Ezekannagha and Georges Hattab for their supervision and assistance.

I also want to thank Leonie Mohr, Yannik Juraschek, Colin Krag, my family and all other friends and colleagues for their emotional support during the writing of this thesis.

Abstract

With the rise of big data in recent years, the need for reliable and space-efficient data storage solutions has also increased. One solution to this is storing data in synthetic Deoxyribonucleic acid (abbreviated DNA). Although this solution is robust, errors occur during long-time storage. However, in the presence of too many errors, it can become difficult to retrieve the original data.

Motivated by the need to identify and understand the errors that occur during storage, this work provides an interactive visualization tool called DeBruVis. DeBruVis provides the interface that allows the analysis of stored sequences. To achieve this, a graph structure called the De Bruijn Graph was used to provide a visual representation of sequences. The visual representation allows the comparison of stored sequences keeping track of the presence of errors.

To show the applicability of DeBruVis, two use cases are proposed using simulated aging and Dorn data set. In addition, to evaluate the tool, a performance analysis on different configurations was reported.

The results of these evaluations led to the conclusion that DeBruVis allows the identification of errors and visualization of stored sequences for relatively small sets of sequences of a couple hundred bases, however further work is needed to support the error identification more clearly and to be more justifiable to use for more and longer sequences. In sum, more research is needed to justify the use of De Bruijn Graph as a suitable visual structure for DNA sequence visualization.

Contents

1	Introduction	9
1.1	Background and Motivation	9
1.2	Objectives	12
1.3	Structure	12
2	Related Work	13
2.1	DNA Data Storage	13
2.2	Errors appearing during long-term storage of data in DNA	14
2.3	De Bruijn Graphs	14
2.3.1	Compact De Bruijn Graphs	15
2.3.2	Colored De Bruijn Graphs	17
2.3.3	Visualization of De Bruijn Graphs	17
3	Materials	19
3.1	Design Principles	19
3.2	De Bruijn Graph Visualization	19
3.3	Data	20
3.4	Packages	21
3.4.1	pandas	21
3.4.2	random	21
3.4.3	networkx	22
3.4.4	pyvis.network	22
3.4.5	Levenshtein	22
3.4.6	matplotlib.pyplot	23
3.4.7	streamlit (and child packages)	23
3.4.8	tempfile	23
3.4.9	time	23
3.4.10	textwrap	23

4	Methods	24
4.1	Back-end Implementation	24
4.1.1	Data Reading	25
4.1.2	Data Pre-Processing	26
4.1.3	Extracting Sequences from the Data	26
4.1.4	Converting Sequences to Lists of Kmers	26
4.1.5	Comparing Sequences	27
4.1.6	Retrieving Nodes and Edges	28
4.1.7	Optional: Compacting Nodes	28
4.1.8	Building the De-Bruijn-Graph	29
4.1.9	Visualizing the De Bruijn Graph	29
4.2	Frontend Implementation	29
4.2.1	Errors between Sequences Graph	30
4.2.2	Different Kmers per Index	30
4.2.3	Streamlit Application	30
4.3	Challenges	31
5	Results	33
5.1	DeBruVis - An interactive visualization tool for DNA data storage .	33
5.1.1	The Data Tab	33
5.1.2	The Statistics Tab	34
5.1.3	The INTRA Tab	34
5.1.4	The INTER Tab	35
5.2	Usecase 1: Simulated Aging Analysis	35
5.3	Usecase 2: INTER Analysis with Differently Encoded Sequences . .	40
5.4	Performance Analysis	41
6	Discussion	51
7	Conclusion	53
	References	56

1. Introduction

The following sections provide some background information about DNA data storage and De Bruijn Graphs. Additionally, the motivation and the structure of the thesis are outlined.

1.1 Background and Motivation

As the era of big data continues to unfold, there has been an unprecedented surge in the demand for dependable and space-optimized data storage solutions. According to Dong *et al.*, the amount of data that has to be stored will reach 175 ZB by 2025 [1]. Current solutions such as tape and discs will be unable to store this amount of data realistically in the future [2]. Additionally, the silicon used in other storage devices polluted the environment, and the deposits holding it will be depleted one day. However, DNA, which holds the genetic information of humans and other life-forms, has turned out to be a sustainable alternative. When storing the data inside DNA, all the information the world produces in a year could theoretically be stored within a couple of grams of it [3]. DNA data storage is also efficient regarding power usage as working with DNA is a million times more effective than working with a personal computer [3]. Lastly, DNA-stored data is relatively safe from destruction as each sequence can be amplified using a process called Polymerase Chain Reaction (PCR), which serves as redundancy for the data [2]. PCR is a process that uses DNA Polymerase's ability to synthesize a new strand complementary to a given strand. Because of these beneficial factors, the field of DNA data storage has seen a lot of research in the last decades, starting with the first attempts by Clell *et al.* [4, 5]. However, the DNA-data storage system is not rid of challenges. One challenge is the presence of errors that occur at different stages of the process. With the presence of errors in this process, it can be difficult to retrieve the original data.

The process of DNA data storage is divided into six steps, which we will outline

in the following section.

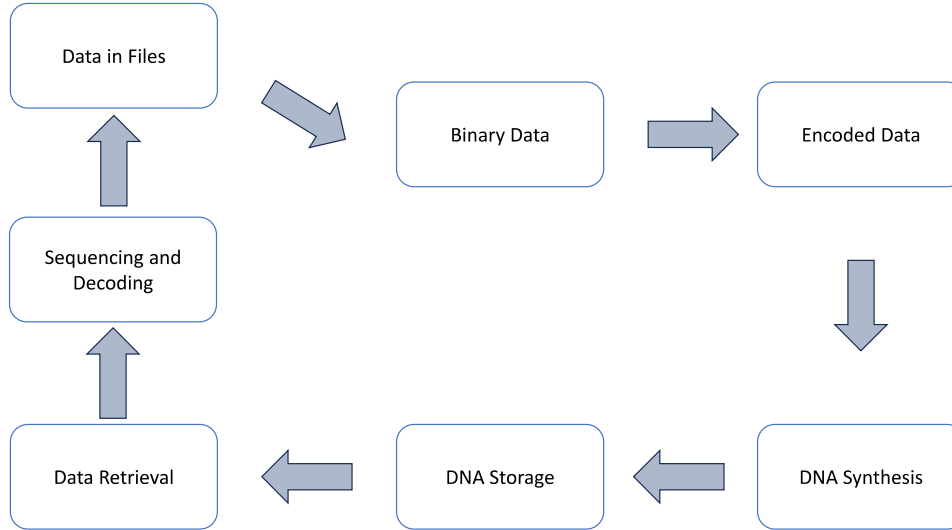


Figure 1.1: An overview of the DNA data storage pipeline.

First, the data is converted into a binary format. Then, it can be encoded into a sequence of bases. Typically, the entire data will not be encoded into a single sequence but rather be broken down into parts and encoded into nucleotide sequences of about 300 bases length [2]. A Base (A, C, G, T) represents one of the four nucleotides Adenine, Cytosine, Guanine, and Thymine. In the third step, the DNA has to be synthesized. This process enables writing these sequences and can be done chemically or enzymatically. The chemical method relies on phosphoramidite chemistry methods [6, 7] and involves repeatedly adding nucleotides at the end of the synthetic DNA, while the enzymatic approach relies on DNA Polymerase. The latter approach saw its first research in 2010 with initially high error rates but has become a fast, efficient, and cost-effective method with low error rates [8, 9]. After synthesis, the DNA data can be stored using either a chemical or a physical method. In the former, the DNA is stored in silica nanobeads and then referenced and distributed in microplates or microtiter plates. At the same time, the physical method uses a stainless steel capsule on the outside and glass on the inside [10,

11]. To retrieve the data later, a selective PCR of the required sample can be used [12]. Lastly, after storage, DNA sequencing can be used to retrieve the data before it is decoded to receive the data.

During certain stages of DNA data storage, one of three types of errors can occur. These three types include Deletion, Insertion, and Substitution and relate to a base missing (Deletion), being added (Insertion), or changing into a different base (Substitution) inside a sequence.

Research on the handling of these errors has already been done with some of it using a graph structure called the De Bruijn Graph, which had previously seen usage in genome assembly, a domain different from DNA data storage. In this research, De Bruijn Graphs are used to regain an original sequence from noisy subsequences or handle errors. Although these approaches can handle errors, there has been no research that tries to visualize when and where they appear to give an understanding of the evolution of the data during storage. To comprehend the alterations introduced into DNA-stored data during the storage phase, particularly focusing on error occurrence, we see the need for an interactive visualization tool that, given the stored data at different time points, can compare these time points and help identify, where and when these errors appear.

In order to explain the concept of a De Bruijn Graph, we first have to introduce the Eulerian cycle [13]. A graph has an Eulerian cycle if it is balanced and vice versa. A graph is balanced if the indegrees equal the outdegrees for each node. The question that De Bruijn Graphs answers is: Find the shortest circular superstring that includes all substrings over a given alphabet. In De Bruijn Graphs, these substrings are k -mers. A k -mer is a k letters long sequence with the letters, in this case, being our four bases. There exist n^k k -mers within an alphabet of n letters. In a De Bruijn Graph B given a number k , every $k - 1$ -mer is assigned to a node. Two of these nodes are then connected with a directed edge if there is some k -mer whose prefix is the former and whose suffix is the latter. The edges of B then represent all possible k -mers and an Eulerian Circle in B represents the shortest superstring which includes each substring exactly once.

1.2 Objectives

This thesis aims to design and develop an interactive visualization tool. This tool exploits the potential of the De Bruijn Graph, traditionally employed in genome assembly, to represent and track alterations over time in DNA-stored data, especially when sequence errors occur. Given the novel nature of this approach, the objectives of the thesis are to achieve the following:

First, this work demonstrates the value of employing De Bruijn Graphs for visualization purposes and within the DNA data storage context. Second, in addition to creating the tool, this work validates its utility by visualizing a DNA-stored data set and illustrating its capabilities. These capabilities pivot two key steps, following an ‘overview first, details second’-approach: First, the user gets an overview of the errors and their positions before going into detail by visually comparing sequences to each other. Third, a thorough performance evaluation of the tool’s back-end is conducted, testing its limits concerning parameters such as sequence quantity and length. Fourth and last, the thesis will delve into a discussion on the front-end limitations in terms of visual quality.

1.3 Structure

The thesis starts by presenting the related work regarding DNA Data Storage and De Bruijn Graphs while also discussing different variants of De Bruijn Graphs and their previous usage in visualization. Following the Related Work, we will present the Materials, including the data and the existing Python packages used for the code alongside another tool called ‘MESA’ that we used to simulate aged DNA data sets. Afterwards, the Methods chapter will explain the Implementation of both the back-end and the front-end of the thesis with a special emphasis on the data processing pipeline. The Results chapter will first describe the front end of the tool, then present two use cases of a simulated DNA data set and a time performance analysis. After that, we will discuss these results before drawing a conclusion and setting a view for possible future work.

The github repository for this thesis, including the source code, usecase data and results can be found under <https://github.com/FloSchwa/MastersThesisDeBruVis>.

2. Related Work

This chapter will give an overview of the research related to our work. It introduces DNA data storage and errors appearing during it. Afterward, De Bruijn Graphs and the two different representations of it, whose concepts we want to use, are presented. Then, we will briefly mention De Bruijn Graphs in DNA data storage before concluding with the current field of research in the visualization of De Bruijn Graphs.

2.1 DNA Data Storage

As already mentioned in section 1.1, data storage in DNA has seen a lot of research in recent years. The first attempts at this were done in 1999 by Clell and Risca and Bancroft [4]. They used it during the second world war to encrypt data in written letters by using the four bases, PCR primers, and an encryption key in the form of base triplets for each letter in the English alphabet and Arabic numbers. This experiment showed that DNA can be used to store data in an encrypted format. In 2001, Bancroft, Bowler, Bloom, and Clelland continued in this field [5]. They defined iDNA (information DNA - the encoded data) and the Poly Primer Key (the primer base sequence to access the iDNA). They used amplification by PCA and thus gained fragments of iDNA that, when analyzed together, yielded the entire information. With this approach, they encrypted Charles Dicken's 'A Tale of Two Cities' opening lines. Although this was the first time a DNA-based data storage mechanism was used, it was done under laboratory conditions and thus didn't consider the safety and tolerance of outside factors.

Up until 2009, data was encrypted in base triplets. However, a proper encoding scheme was necessary [3]. Following this, encoding schemes like Huffman implemented [14]. In the 2010s, several projects drove this research forward, and the amount of data encrypted went from kilobytes to megabytes [15]. Another break-

through was achieved by Goldman *et al.* in 2013, who provided a scalable method that would become cost-efficient within a decade and, most importantly, found out that the decrease in encoding efficiency is very slow when data size increases [16]. The challenges remaining at this point were the slow access and retrieval rates and time-consuming encoding, amplifying, sequencing, and decoding, which were researched within the following years.

2.2 Errors appearing during long-term storage of data in DNA

At different stages in the process of DNA data storage, errors can appear in the sequences. Apart from errors during synthesis and sequencing, they can also happen during storage [17]. Outside factors can have a significant impact on DNA data storage. According to Xu *et al.*, high humidity, light, and heat can cause damage to DNA molecules. Also, DNA strands have to be immobilized in some cases, so the linkage chemistry also impacts the system's robustness [18]. Another cause of errors can be the chemical decay of the DNA, especially hydrolytic damage, as mentioned by Heckel *et al.* [17]. This can cause depurination, leading to strands breaking and deamination, the change from cytosine to uracil in the C-G base pair. Depending on the enzymes used in PCR, there is a different effect on the result. When using proofreading enzymes, the amplification reaction stalls at the U nucleotide, and no sequence compromising a U nucleotide is formed. Since these incomplete DNA sequences are not further amplified in the next PCR cycle, they are diluted out due to the lack of a primer on both ends of the sequence, while the G-strand is still amplified further. However, when using non-proofreading enzymes, the sequence with the U-G base pair is half amplified correctly to a C-G base pair and half amplified incorrectly to a T-A base pair, resulting in an error [17].

2.3 De Bruijn Graphs

The idea of De Bruijn Graphs started in 1946 when Nicolaas De Bruijn attempted to solve the 'superstring-problem': Find the shortest circular superstring that contains all possible substrings of length k (also called k -mers) over a given alphabet [13]. His solution, the De Bruijn Graph, has been discussed in the 1.1. It should be noted that the De Bruijn Graph has mainly been used for genome assembly,

rather than for DNA data storage. However, Song et al. showed in 2020 that this approach is viable and presented a robust and efficient algorithm [19]. They used it for error handling, more specifically for handling base errors, which will not be the main focus of this thesis. Additionally, Kiah et al. uses De Bruijn graphs to reconstruct sequences from noisy sub-sequences while retrieving data from DNA [20].



Figure 2.1: An simple example of the corresponding De Bruijn Graph for the sequence AACCGGGTA.

2.3.1 Compact De Bruijn Graphs

Compact De Bruin Graphs apply an additional compaction step to the graph as data reduction. According to Chikhi et al., it is the bottleneck to many genome assembly pipelines, which is why they provided an efficient algorithm for it in 2016 [21]. The idea of compact De Bruijn Graphs lies in the ‘compaction’ of paths within the De Bruijn Graph. In the first step, sometimes called k -mer counting, all unique k -mers within the reads are counted. In the second step, all unitigs, which are paths

with all but the first vertex having an in-degree of one and all but the last vertex having an out-degree of one, are compacted into a single vertex [13]. Furthermore, Rodland provided another efficient data structure: a searchable index of all substrings from a set of strings [22].

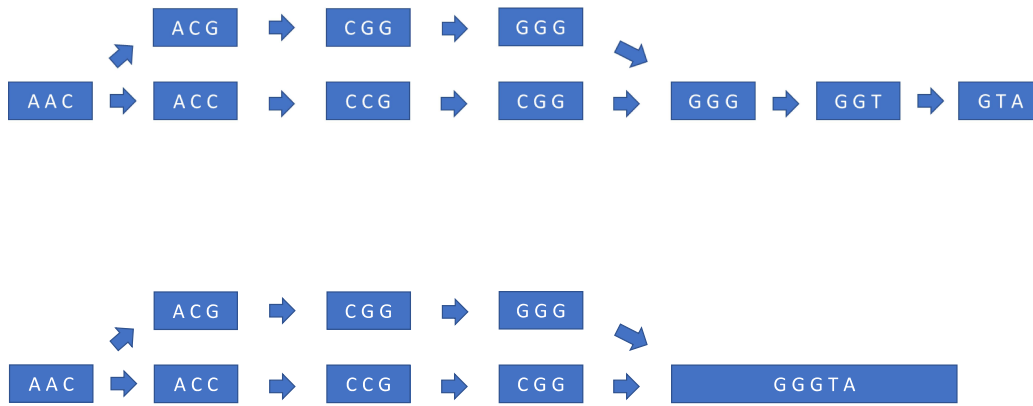


Figure 2.2: An example of a compacted De Bruijn Graph in our understanding. All kmers that are common between all sequences are compacted.

Within the context of related work, the De Bruijn Graph variant applied in this study has been traditionally used not for visualization but for enhanced computational efficiency. Nevertheless, the minimization of the resultant graph’s visual space might offer a beneficial interaction element for our execution. In De Bruijn Graphs, errors can be discerned as branches. Hence, paths within the graph that do not branch off - in other words, devoid of errors, and precisely termed ‘unitigs’ - are less compelling and can consequently be consolidated.

2.3.2 Colored De Bruijn Graphs

The concept of Colored De Bruijn Graphs was first introduced by Iqbal et al. in 2012 and later enhanced by Muggli et al. in 2017. Muggli et al. contributed an efficient data structure for these Colored De Bruijn Graphs, as outlined in their work [23, 24]. According to Iqbal et al., the aim is to facilitate ‘detecting and genotyping simple and complex genetic variants in an individual or population’ [23]. While the edge structure mirrors that of the conventional model, each vertex and node in a Colored De Bruijn Graph is allotted a distinct color. The color assignment depends on the presence of the specific vertex or node within the samples.

Comparable to the Compact De Bruijn Graphs, Colored De Bruijn Graphs have been primarily employed to augment computational efficiency, not as a visual architecture. Nevertheless, enhancing the graph with an additional visual dimension by coloring its edges could assist in differentiating the samples within the resulting graph. Hence, we believe that this version of the De Bruijn Graph warrants exploration and utilization in our study.

2.3.3 Visualization of De Bruijn Graphs

Since this thesis aims to implement a visualization tool for errors within sequences of DNA-stored data, De Bruijn Graphs of possibly huge sizes must be rendered in real-time. Tools for this have already been implemented, such as 3DScapeCS [25]. Additionally, Andri Shostatskyi provides a tool for visualizing De Bruijn Graphs, but no publication is attached to it. It is worth mentioning that De Bruijn Graphs have not been used for a visualization application, making our approach novel. Current research uses them exclusively for computations, handling errors, or finding the correct sequence among a set of sequences. However, we still believe that there is merit in using them to visualize errors and to help understand how sequences evolve during long-time storage.

3DScapeCS is a free open-source platform provided by Wang et al. in 2013 [25]. It builds on a different model, Cytoscape, and can convert 2D graphs from Cytoscape to 3D graphs while being, other than Cytoscape, able to show thousands of nodes and edges instead of just hundreds [25]. This is due to the layout building algorithm, which has a complexity of $O(n^3)$ in Cytoscape while being $O(\log(n))$ in 3DScapeCS. Additionally, it keeps all colors, sizes, and shapes of all nodes and edges

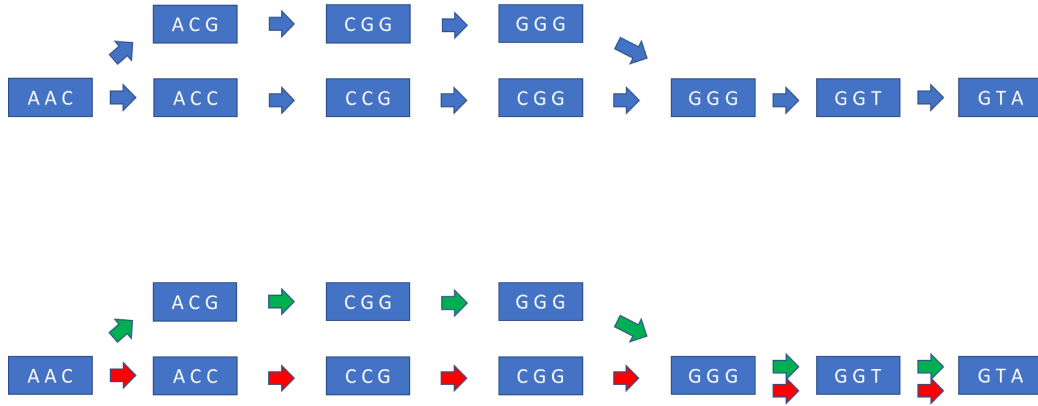


Figure 2.3: An example of our understanding of a colored De Bruijn Graph. Edges are colored based on the sequences they come from.

through the conversion. Lastly, 3DScapeCS can use a plugin called VistaClara to display the changes over time from multiple experiments, which could be useful for our tool [26].

Although 3CScapeCS can visualize De Bruijn Graphs as well as other graphs, it is a separate program with its front-end. This will make it impossible to link it to a visualization front-end like the one we want to provide. Because of this circumstance, we will implement the back-end of building De Bruijn Graphs, handling them efficiently and visualizing them while providing the visual characteristics of both Colored and Compact De Bruijn Graphs in a front-end application by ourselves.

3. Materials

This chapter covers the design principles of the interactive visualization tool, explains how it visualizes DNA data storage with the help of De Bruijn Graphs, and discusses the software and technologies used.

3.1 Design Principles

The interactive visualization tool allows users to explore data stored in DNA using De Bruijn Graphs and identify errors. The underlying story should be: ‘How did the DNA-stored data set evolve over time?’. Users should be able to upload their data. This thesis aims to provide a tool that follows an ‘overview first - details second’ approach by first providing users with an understanding of the errors within the data by showing them how many errors there are and how they are distributed. Afterward, we give them the means to explore the change of the sequences visually and in detail by comparing them to each other using De Bruijn Graphs. The graphs should always be colored; compaction should be possible if the user chooses. The graphs should be interactive and support filtering, dragging, and zooming since we expect them to possibly become large. Because of the possible size of the graphs, the main part of the screen should be dedicated to them to give users the best readability possible.

3.2 De Bruijn Graph Visualization

The tool mainly uses a combination of the Python packages `pandas`, `networkx`, and `pyvis.network` to visualize the De Bruijn Graphs. `Pandas` handles the tables providing the edges and nodes and building them from the sequences, while `networkx` uses these nodes and edges to build the graphs. Finally, `pyvis.network` produces an HTML webpage with the interactive visualization of the graph from `networkx` and already has built-in zooming and dragging.

3.3 Data

To generate the data to demonstrate the sequence comparison capabilities of our interactive visualization tool, we used ‘MESA’, a tool developed by Schwarz et al. in 2020 at the University of Marburg [27]. Given a sequence, MESA can simulate how the sequence would change over time with certain parameters like temperature. MESA is a web application that is freely accessible and can also be accessed via docker or github. We used the web version (<https://mesa.mosla.de/>). At first, users have to upload data as either a sequence, a FASTA-file, or a configuration. Afterward, they can choose between a wide range of parameters like the Kmer windows, the synthesis method, the number of PCR cycles and PCR Polymerase used, and the months of storage to be simulated. To generate the data, we used the Python package ‘random’ to receive a sequence of 160 bases in length and saved it inside of a FASTA file. Then, we uploaded this file to MESA. We chose the standard parameters of the tool and generated three aged data sets after 10.000, 25.000, and 40.000 years to represent long-time storage. Finally, the resulting aged sequences can be downloaded from MESA as FASTQ files. Since our visualization tool reads the data from a single FASTA file, we used the standard Python file-reading capabilities to read the three FASTQ files. We converted them into a merged FASTA file using the standard Python file-writing capabilities. The resulting FASTA file holds the sequence at all three time points and can subsequently be read in by our visualization tool.

Additionally, this thesis uses the dorn data set to demonstrate the comparison between sequences of different encodings. The dorn data set is the story of dorn-röschen, stored in DNA. As the two different encodings, fountain code, and grass code are used [28, 29].

3.4 Packages

Several already existing Python packages were used to implement the methods found in this work. This section will give an overview of which packages were used and how they were deployed.

package name	version	description
pandas	1.1.1	Data reading, writing, and manipulation
random		Generation of random samples
networkx	3.0	Building graphs from lists of edges and nodes
pyvis.network	0.3.2	Interactive visualization of graphs
Levenshtein	0.20.9	Computing Levenshtein distance
matplotlib.pyplot	3.3.1	Graph plotting
streamlit	1.22.0	Building front-end web application
streamlit.components.v1	1.22.0	Importing html-files into web application
streamlit.optionsmenu	0.3.2	Including a sidebar in the web application
tempfile		Generating a named temporary file
time		Returning current time
textwrap		Divide a string into packets

Table 3.1: Overview of the packages used in this thesis.

3.4.1 pandas

Pandas is a Python tool for data analysis and manipulation [30]. Its primary data structure is the data frame. Dataframes are two-dimensional, tabular structures with labeled axis for rows and columns. They appear in most functions of this thesis and hold information about the nodes and edges alongside their properties like the color or label of the De Bruijn Graphs.

3.4.2 random

Random is a Python package that is used to generate random samples from a given alphabet. The thesis used it to generate random test samples of a given length from the alphabet of nucleotides.

3.4.3 networkx

Networkx is a Python tool for building graphs from nodes and edges [31]. The nodes and edges can be read from pandas data frames. After defining the data frames for nodes and edges, we generate an empty graph from networkx. The type of graph can be chosen from a wide array of different graphs, of which we choose a MultiDiGraph. A MultiDiGraph is a directed graph that supports multiple edges between nodes. This is necessary because nodes (which correspond to Kmers in the sequence) can be shared between multiple sequences if the same Kmer is present at the same index in all sequences. After the empty graph is defined, nodes and edges can be added from the rows of their respective Data Frames. In addition, networkx supports additional parameters to the edges and nodes, such as color and weight. Because of this, we color the edges based on the sequence they belong to and assign a weight of 4 to each edge, which we deemed an appropriate size for readability. The assigned colors follow the 'Tableau 10' color scheme to ensure they can be distinguished by users with color deficiencies.

3.4.4 pyvis.network

Pyvis.network is a Python tool for the interactive visualization of graphs [32]. It can read graphs generated with networkx and yields an HTML-webpage which shows the graph and provides interactivity such as dragging and zooming. Additionally, a function called `show_buttons` within pyvis.network can give users many customization options, such as physics and graph layout. However, we decided against opening up this functionality for the users of our tool since it is likely to overwhelm them.

3.4.5 Levenshtein

The Levenshtein package within Python provides functions that are related to the Levenshtein distance. The Levenshtein distance between two strings is defined as the minimum number of string operations (deletion, insertion, or substitution) that are necessary to change one string into the other [33]. In this thesis, we use it to compute string similarity and determine which errors are present when two sequences are compared, which is a measure that can be displayed in our tool.

3.4.6 matplotlib.pyplot

Matplotlib.pyplot is a Python package that can be used to generate charts from data [34]. In our tool, two charts are used: A bar chart that displays the errors of each of the three types between sequences and a line chart that shows the number of different Kmers on each index in the De Bruijn graph to give users an idea of how crowded certain areas of the resulting graph will be.

3.4.7 streamlit (and child packages)

Streamlit is a Python tool for building web applications. We use it to build the front end of our tool in the form of a multi-page web application. A multi-page web application has a sidebar from which users can navigate to different pages. Given the potential size of the graphs that will be shown, it is impossible to have all functionalities of our tool on a single page. Streamlit supports multiple components, from which we use buttons, selection fields, checks- and textboxes alongside headers, subheaders, and charts—using the components.v1 library, HTML content such as the interactive graphs from pyvis.network can be displayed. Lastly, we store all user selections within a session so they can freely switch between the tabs.

3.4.8 tempfile

This thesis uses the Python package tempfile to create a temporary file of the data the user uploads into our tool as a working copy.

3.4.9 time

To report on the runtime performance of important functions, the time package was used. Its function `time()` can display and store the current time. Calling it before and after running a function and subtracting the values yields the runtime of the function.

3.4.10 textwrap

This package and its function `wrap()` was used to divide a sequence into packets of a given length.

4. Methods

This chapter presents the process of implementing both the back-end and front-end of this thesis and the evaluation. It also presents the challenges that occurred during implementation and how they were solved.

4.1 Back-end Implementation

The first section details the implementation of the back-end. Specifically, it provides insights into the structures that were used and how they were used. The following section will be divided into nine steps, which outline the processing pipeline from the DNA-stored data to the interactive visualization of the De Bruijn Graph.

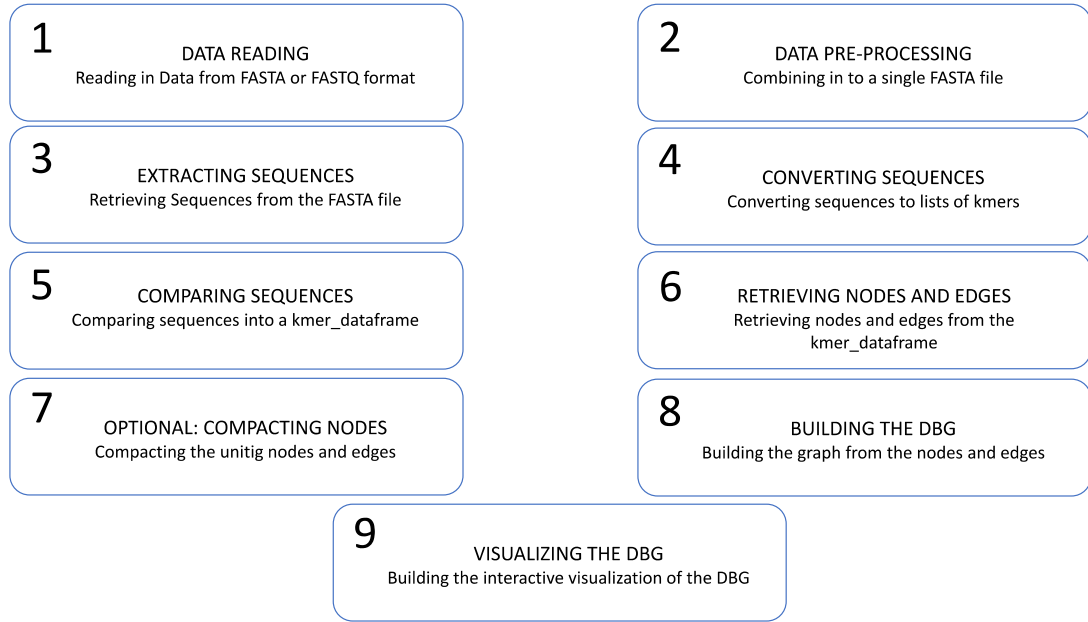


Figure 4.1: An overview of the pipeline, starting from the FASTA or FASTQ file and ending in the interactive visualization of the De Bruijn Graph.

4.1.1 Data Reading

Before commencing data processing, the initial step involves loading the data into the tool. As the tool is designed to read only a single FASTA file, data processing varies depending on its format. For this study, the data is either in FASTA format or, in the case of MESA-originated data, FASTQ format. As part of this research, dedicated functions are provided to read both of these types.

If a sequence is stored in FASTA format, it is segmented into packets. Each packet subsequently receives a number and is stored in a text-like file. Lines within this file alternate between the identifier of the sequence, which is preceded by a '>', and the sequence packet. Upon reading sequences from the FASTA file, the tool processes it from the beginning to the end. If a line initiates with '>', it is an identifier line; otherwise, it is the corresponding sequence to the last identifier. These lines are then preserved in a dictionary. A dictionary is a two-dimensional structure comprising keys and values. In this context, the keys are the identifiers,

and the values are the corresponding sequences. These sequences can later be retrieved by referring to their respective key.

In the case of data in FASTQ format from MESA, it is also stored in a text-like file. However, the first line links to MESA, and the third line consistently represents the entire sequence. Consequently, the tool reads the third line to obtain the complete sequence.

4.1.2 Data Pre-Processing

Given the tool's design to process a single FASTA file, some pre-processing must be applied to the data. To reemphasize the objective of this tool, it is designed to compare data at multiple time points, with each time point potentially represented by its individual FASTA or FASTQ file. This necessitates the merging of FASTA files. This process involves reading each FASTA file as outlined in 4.1.1 and modifying their identifiers by appending a time point suffix, indicating their respective timeframe. If the need arises to merge multiple FASTQ files, each FASTQ file is read as described in 4.1.1, assigned an identifier suffixed with a time point, and all sequences are preserved in a unified FASTA file.

4.1.3 Extracting Sequences from the Data

Once all sequences have been stored in a singular FASTA file, they can be extracted using Python's inherent file-reading functionalities. The FASTA file is read line by line, mirroring the approach outlined in 4.1.1, and stored within a dictionary. Consequently, individual sequences can be accessed by referencing their unique key identifier in the dictionary.

4.1.4 Converting Sequences to Lists of Kmers

Following the retrieval of sequences, they are converted into lists of their respective Kmers. For a specified K , the initial Kmer of a sequence is denoted by the first K bases within that sequence. Consequently, the second Kmer is a K -length subsequence commencing at the second base, and so forth. This process generates a list with $l-(K-1)$ Kmers for a sequence of length l , which, along with their indices, are preserved for subsequent steps. Henceforth in this section, any reference to a sequence implies the Kmer-list corresponding to that sequence.

4.1.5 Comparing Sequences

The next step in the pipeline entails the comparison of sequences to identify discrepancies between them, later visualized in a De-Bruijn-Graph. This comparative process, involving multiple sequences, uses a comparison of two sequences as its base. This comparison between two sequences is then performed between one reference sequence with each other sequence in the set.

When comparing two sequences, two steps are executed: First, a supporting structure termed as a `conflict_dataframe` is constructed, followed by the creation of a so-called `kmer_dataframe` using this `conflict_dataframe`.

To generate the `conflict_dataframe`, indices are compared at each index, returning a 0 if the Kmer at that index is identical across both sequences and a 1 if the kmers at that index differ. If one sequence exceeds the other in length, a 1 is automatically returned for each index following the termination of the shorter sequence. Alongside these 0s and 1s, a structure termed as a `streak-list` for the comparison is constructed, comprising index ranges where the two sequences align, later utilized to identify the unitigs for compaction.

Utilizing this `conflict_dataframe`, the `kmer_dataframe` is determined. This `kmer_dataframe` will eventually become the list of nodes for the De Bruijn Graph. Consequently, information such as the sequence from which the kmer originated and whether it matched the other kmer at the same index is preserved. With this in consideration, the `conflict_dataframe` is iterated over and an identifier is assigned to each kmer, based on its originating sequence. This identifier is a numerical value incrementing from zero. Subsequently, it is appended to the `kmer_dataframe` with its identifier, its index, and the kmer. If the `kmer_dataframe` reported a 0 at that index, indicating it exists in both sequences, it is added twice, once for each identifier. However, if the `kmer_dataframe` reported a 1 at the index, it is only added once, with the identifier of the sequence it was found in.

If more than two sequences are compared, the first sequence is established as the reference, and each subsequent sequence is compared to it, following the aforementioned procedure. For each sequence beyond the initial two, the identifier variable is incremented, ensuring each sequence possesses its identifier. Presuming a comparison involving n sequences, the final sequence will therefore have the identifier $n-1$.

4.1.6 Retrieving Nodes and Edges

Upon obtaining the `kmer_dataframe`, the next step involves preparation for the construction of the De Bruijn Graph. This is achieved by first defining the edges of the forthcoming graph, followed by the determination of nodes. This process necessitates manipulating the previously obtained `kmer_dataframe`. For successful graph construction, each kmer (which will serve as nodes in the ensuing graph) must have a unique index that delineates its position and neighboring kmers. As a result, an iteration over the rows of the `kmer_dataframe` is initiated.

For each index, if more than one unique kmer is present at that index, it indicates non-identical sequences at that position. Consequently, each of them may require the assignment of a new index. Under this circumstance, one of two situations may arise: If it is the first occurrence of non-identical kmers at an index, the first kmer at that index can retain its index. However, for all other kmers at that index, an index shift must be implemented to ensure its uniqueness. This shift is maintained throughout the iteration and incremented each time a kmer is assigned a new index.

Alternatively, if kmers at an index are identical, then the sequences at that index were the same. In this scenario, there's no need to shift the index for kmers with the same index. However, if non-identical kmers were encountered previously and a shift is already in place, the index must be set to that of the last kmer and incremented by one to update it.

Following this methodology, an updated `kmer_dataframe` is obtained with unique indices for each individual kmer. This updated `kmer_dataframe` will form the foundation for the edges in the subsequent step. Additionally, to procure the nodes, a copy of the `kmer_dataframe` is created and all nodes that duplicate at both index and kmer are removed. This operation is accurate because if a kmer is present multiple times at an index, it implies that it existed in several sequences at that index. Consequently, these entries will share a node in the subsequent stages.

4.1.7 Optional: Compacting Nodes

If the graph requires compaction, the indices also necessitate subsequent updating. This is accomplished by initially consulting the `streak_lists` that were computed during the conflict report. Given that each comparison of two sequences produces its own `streak_list`, the first step is to amalgamate them. This is achieved by calculating their intersection, resulting in a list comprising solely the indices whose kmers are identical across all sequences.

The `kmer_dataframe` is then utilized once more, iterating over its rows. For each index, if it is present in the combined `streak_list`, the kmers at that index and every subsequent index until an index not present in the `streak_list` is reached must be compacted. Compaction is performed by reverting the kmers back into their sequence, progressively adding the last base of the kmer to the first kmer scheduled for compaction. Simultaneously, the index of the last kmer for each compaction is stored. This is essential as the indices need to be updated again due to the removal of kmers (and their associated indices). Finally, the rows with the now compacted kmers are replaced with the compacted kmer.

To update the indices, shifts akin to those from the section on retrieving nodes and edges are implemented. If a kmer at an index has been compacted, the number of kmers it was compacted from is subtracted from its index. Additionally, this shift must also be applied to all subsequent kmers.

4.1.8 Building the De-Bruijn-Graph

Once the nodes and edges have been ascertained, it becomes possible to construct the De Bruijn Graph. Initially, an empty `MultiDiGraph` is generated (as referenced in the section on NetworkX). Subsequently, nodes are added at their respective indices. Following this, the edges for each sequence are established. For each sequence identifier in the `kmer_dataframe`, the data frame is filtered by it. Then, an iteration is performed over the kmers (now identified by their nodes) in the sequence, and they are added one at a time. Given the accuracy of the indices ensured in the prior steps, this procedure can be executed with relative ease.

4.1.9 Visualizing the De Bruijn Graph

The concluding step involves constructing the interactive visualization for the De Bruijn Graph. Leveraging the package that supports NetworkX graphs allows for easy interpretation of the graph from section 4.1.8. As a result, an interactive graph is rendered as an HTML application.

4.2 Frontend Implementation

After laying out the back-end implementation in 4.1, the implementation of the front-end can be detailed. Beforehand, two functions are implemented in the back-end but are not part of the pipeline. These functions return graphs for the tool

and will commence this section before continuing with the implementation of the Streamlit-application. The Streamlit-application, which is the front-end of our tool, is divided into four parts: The data upload, the statistics, the analysis between single sequences (further called INTRA analysis), and the analysis between whole sets of sequences (further called INTER analysis). These four parts will outline the rest of this section.

4.2.1 Errors between Sequences Graph

The graph resulting from this function displays, for a set of sequences, how many of each type of base error appear when comparing the first sequence in the set with each other sequence in the set. It was implemented using the package Levenshtein (as presented in 3.4.5), which is capable of reporting the quantities of each type of error between two sequences using the Levenshtein distance. Using this function, the overall number of errors alongside the quantity for each type of error is returned. These values will be displayed in a bar chart.

4.2.2 Different Kmers per Index

This function returns the number of different kmers for each index. This measure is determined by summing up the different kmers per index after the ‘Comparing sequences’ step of the pipeline before the indices have been updated.

4.2.3 Streamlit Application

As previously described in 4.2, the Streamlit application has four parts, whose implementation will be detailed now. Beforehand, the Streamlit application is defined as a multipage application, so that each of the four parts can have its own page, and switching between them is supported. So-called ‘session-states’ are defined to store variables when switching between pages.

4.2.3.1 Data Upload

The data upload has four components. First, a data upload component from Streamlit that can read FASTA files and handle them via the function presented in 4.1.1. Second, an integer selection field for the kmer-size. Third, a checkbox component from Streamlit opens up an additional selection field for a second set of sequences for INTER analysis if enabled. Fourth, a selection field from Streamlit allows the

selection and storage of any number of sequences from the data uploaded in the first component for later use. Lastly, if the checkbox in component two is enabled, a second selection field that again allows the storage of any number of sequences from the uploaded data.

4.2.3.2 Statistics

The statistics tab again has four components. First, a checkbox that allows the user to toggle between the error statistics of only two sequences or a set of sequences. Second, if the checkbox is unchecked, a selection field for two sequences from the selection made in component three of 4.2.3.1 that then displays a message with the quantities for all errors and then all types of errors between the two sequences. Third and fourth, if the checkbox is checked, a selection field for any number of sequences from the selection made in component three of 4.2.3.1 that then displays the bar chart from 4.2.1 and 4.2.2.

4.2.3.3 INTRA analysis

This tab has three components. First, a selection field for selecting any number of sequences from the selection made in component three of 4.2.3.1. Second, two buttons that start the pipeline are presented in 4.1, either with compaction or without compaction. Lastly, a window for HTML content then displays the resulting interactive graph.

4.2.3.4 INTER analysis

Finally, this tab is almost identical to the one in 4.2.3.3 but has two selection fields and two windows for HTML content to display the two interactive graphs.

4.3 Challenges

The main challenge lies in the assurance of the correct indices at all times for the `kmer_dataframes`. The shifts for both the preparation of the graph and the compaction depend on the constellation of the sequence. Multiple cases must be taken into account as there is a difference between a common kmer being at the very start of the sequence, in the middle, or at the end. The same applies to the kmers that have to be compacted. All these cases were tested, and correct indices

were assured. Another issue, even if it was minor, lay in the conception of the structures and making sure that, for example, a kmer appearing multiple times within a sequence is still seen as a different node later.

5. Results

In the following chapter, we will present the result of this thesis. First, the interactive visualization tool is introduced. Then, two use cases will show its functionalities. Lastly, a performance analysis will give insights into the effectiveness of the tool with varying parameters.

5.1 DeBruVis - An interactive visualization tool for DNA data storage

The primary output of this thesis is the visualization tool named ‘DeBruVis’ (a contraction of De-Bruijn-Visualization). Employing the methodologies delineated in chapter 4, a Streamlit web application has been designed, presenting the project’s backend via a multi-page interface. A multi-page setup has been chosen to prevent the user’s cognitive overload from viewing more than two interactive graphs simultaneously. These diverse pages, hosted as named tabs adorned with task-specific icons in a sidebar, facilitate the following four operations:

5.1.1 The Data Tab

First, in a tab called ‘Data’, a TXT or FASTA file can be uploaded, and the sequences that the user wants to work with can be chosen from the set of sequences in the file. Here, the user can also choose whether to compare single sequences to each other (called ‘INTRA’ analysis) or compare sets of sequences to each other (called ‘INTER’ analysis). Lastly, users can choose the kmer-size for the computations. Mainly, this tab is necessary to import the user’s data and choose the sequences from it they want to explore in the following tabs.

5.1.2 The Statistics Tab

The second tab, ‘Statistics’, enables users to get an overview and a first understanding of the errors, their quantity, and positions. With the help of this statistical overview, they should get an idea of what the graph will look like and whether it is feasible to build it in the first place. In case the sequences have a high number of errors between them, the resulting De Bruijn Graph would branch extensively and be hard to read. If all errors are in about the same area, this area might be interesting to explore but could be cluttered due to the many branches in the respective section of the graph. In addition to these insights, the quantity and type of errors will be almost impossible to read from the graphs, therefore various visualization components have been selected to effectively exhibit this information.

To begin at a small scale, the user can input two sequences out of the set they chose in 5.1.1 and check their similarity, determined by using the Levenshtein algorithm and displaying total errors as well as errors of each type (Substitution, Deletion, Insertion), respectively. If users prefer to explore the entire sets of sequences in the ‘Data’ tab, they can check a box at the top of the page. This exploration of the entire set of sequences allows two more insights. First, a bar chart that displays the types of errors and their quantity, comparing each two sequences from a selection as presented in 4.2.1. Additionally, a second graph in the form of a line chart informs the user about how many different kmers there are at each index as presented in 4.2.2. By evaluating this chart, users can understand how crowded certain parts of the graph are. The lesser kmers there are per index, the fewer branches the graph will have in that area.

5.1.3 The INTRA Tab

In the third and fourth tabs, users can move from the overview they got in the Statistics tab (5.1.2) to a more detailed exploration by referring to the interactive De Bruijn Graphs. At the top of the page, they select a set of sequences from the ones chosen in the Data tab (5.1.1). Furthermore, they can choose between two buttons that build the De Bruijn Graph compacted or without compaction. After clicking this button, the interactive De Bruijn Graph appears below the selection field. With the help of this interactive visualization, we expect users to compare sequences at different time points during storage and identify errors. The compaction can help reduce the visual space of the graph, while the coloring of edges allows them to identify which sequences are branching.

5.1.4 The INTER Tab

Lastly, users can compare two De Bruijn Graphs to each other. To achieve this, they must check the box for INTER analysis in the Data tab (5.1.1) and choose a second set of sequences to build the additional De Bruijn Graph. Afterward, they can return to the INTER tab, possibly choose a subset of their selection in the Data tab (5.1.1) and build the two De Bruijn Graphs. This functionality can, for example, be used to compare two sequences that have been encoded with different encoding schemes.

5.2 Usecase 1: Simulated Aging Analysis

To show the tool's functionalities with an example data set, we generated a DNA data set with MESA (see: chapter 3) and simulated its aging process to get three additional time points of it after 10.000 years, 25.000 years, and 40.000 years. These time points have been chosen after examining the resulting aged sequences for every 1000 years and finding almost no errors up until year 10.000. In the following section, we will use screenshots to visualize the results.

To generate the simulated data set, we first created a test sequence with 160 bases via the method described in 3.4.2, then imported that sequence into MESA. We used the standard parameters except for setting the storage time to 120.000 months, 300.000, and 480.000 months, respectively. Then, to mimic an encoding scheme, we divided the sequence into packets of 16 bases and stored them in FASTA files. Finally, we processed all four FASTA files as presented in 4.1.2 to receive a single FASTA file. Following these steps, the data can be read into DeBruVis as presented in figure 5.1. It should be noted that we don't check the box for INTER analysis since we do not plan to conduct this type of exploration in this use case.

Next, to get an overview of the errors that occur when comparing the sequences, we proceed to the Statistics tab, as seen in figure 5.2. Here, we first let the 'Examine all data' box unchecked and compare some sequence packets of the original data set with the 25.000 years aged data set. After clicking on the 'Compute stats' button, a message is shown that displays the number of total errors alongside the quantities for each type of error. We can see that the comparison introduces only two errors and move on to the overview for error statistics for all sequences by checking the box for the comparison between all sequences.

Figure 5.3 shows the bar chart described in 4.2.1. Note that the comparison between the original sequence and itself trivially has no errors. Furthermore, com-

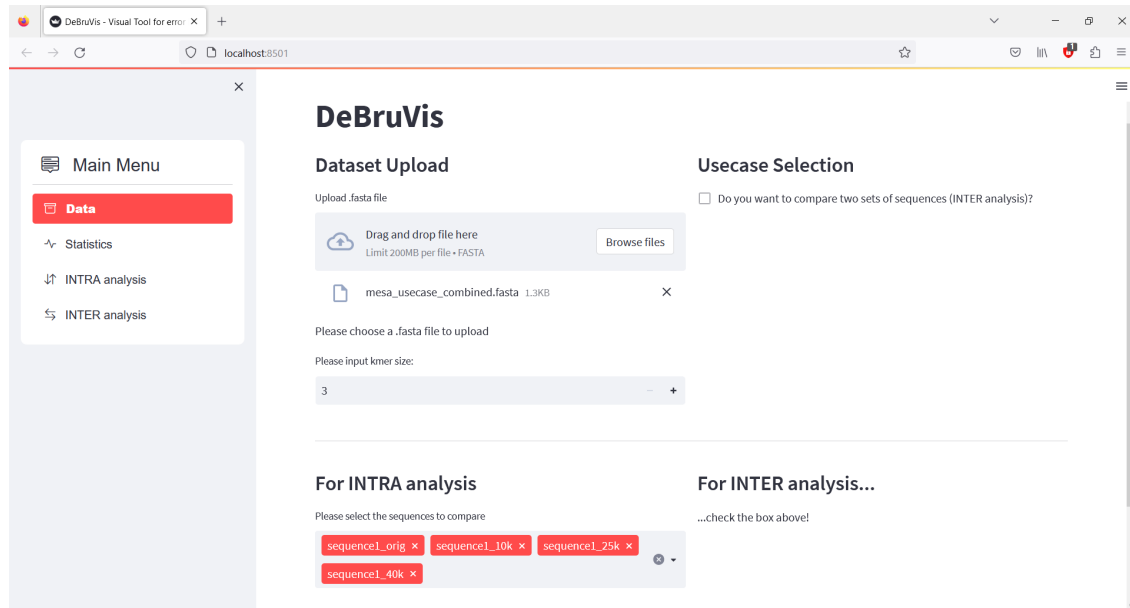


Figure 5.1: Example of the Data Tab with only the INTRA functionality enabled. Note that the selection for the second set of sequences has not yet appeared due to the unchecked box for the INTER use case.

pared to the sequence 10.000 years later, the original sequence has no errors yet.

The line chart in figure 5.4 shows the number of different kmers per index. We can see that the kmers completely align at indices 7 and 11-13 since only a single kmer exists at those indices. At the other indices, we have two different kmers at most.

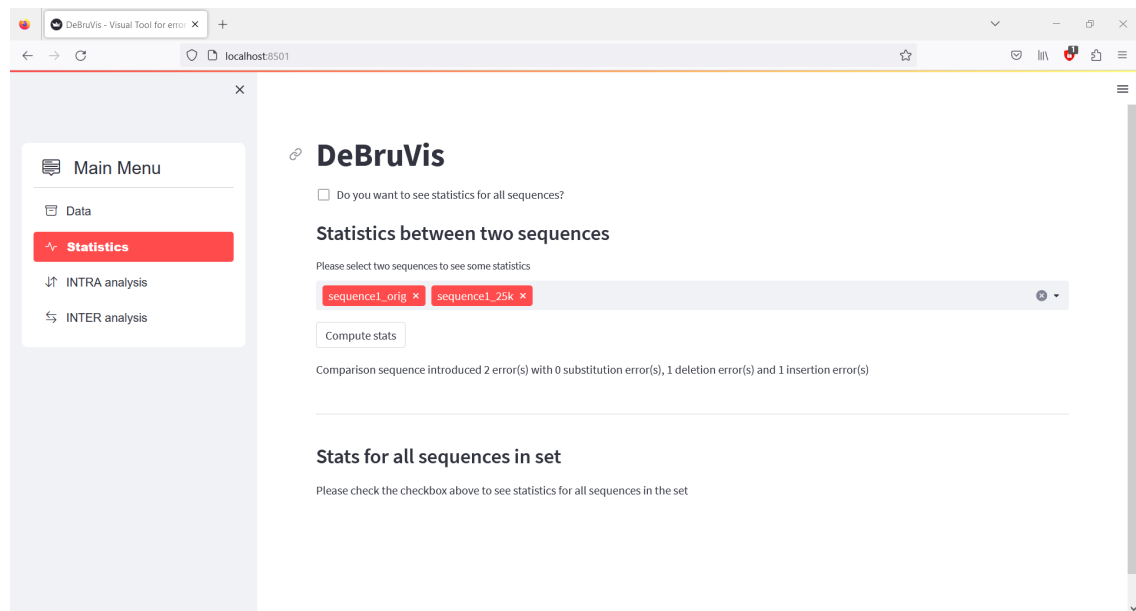


Figure 5.2: Given two sequences in the set, we can see which types of errors and how many appear when the two sequences are compared.

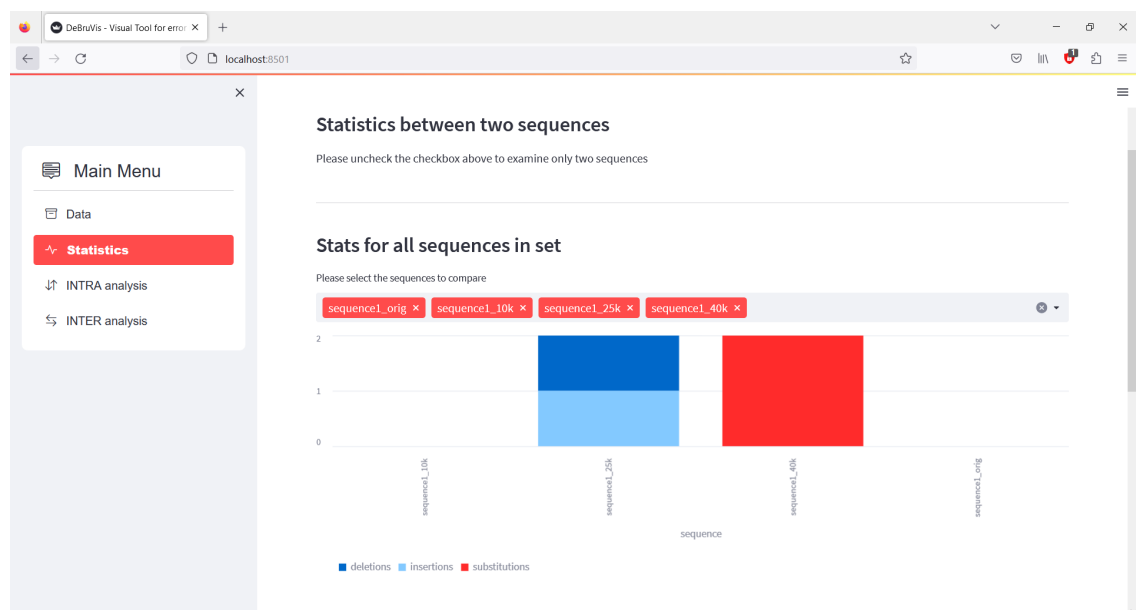
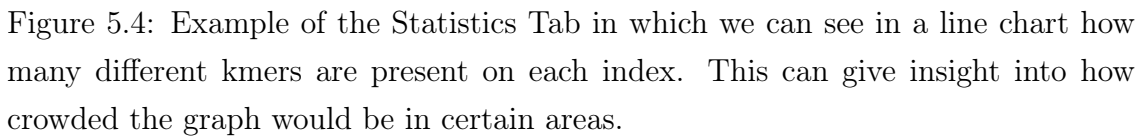


Figure 5.3: This graph within the Statistics tab shows the number and type of errors that comparing every two sequences in the selection introduces.

After receiving an overview of the positions and number of errors, we move into the INTRA analysis tab to compare some sequences and build interactive visualizations. Figure 5.5 shows the non-compacted graph between the original sequence and the sequence after 25.000 years. In case we want to compact this graph, we click the 'Build compacted' button at the top of the page. Figure 5.6 shows this



DeBruVis - Visual Tool for error: X

localhost:8502

Main Menu

- Data
- Statistics
- INTRA analysis**
- INTER analysis

sequence1_orig x sequence1_25k x

```
graph TD; ATA1((ATA)) -- blue --> TGA1((TGA)); TGA1 -- blue --> GAT1((GAT)); GAT1 -- blue --> GCA1((GCA)); GCA1 -- blue --> CGC1((CGC)); CGC1 -- blue --> GCG1((GCG)); GCG1 -- blue --> GAC1((GAC)); GAC1 -- blue --> AGC1((AGC)); AGC1 -- blue --> TAG1((TAG)); TAG1 -- blue --> ATA2((ATA)); ATA2 -- blue --> TAT1((TAT)); TAT1 -- blue --> TGG1((TGG)); TGG1 -- blue --> AAT1((AAT)); ATA1 -- orange --> TGA2((TGA)); TGA2 -- orange --> GAT2((GAT)); GAT2 -- orange --> GCA2((GCA)); GCA2 -- orange --> CGC2((CGC)); CGC2 -- orange --> GCG2((GCG)); GCG2 -- orange --> GAC2((GAC)); GAC2 -- orange --> AGC2((AGC)); AGC2 -- orange --> TAG2((TAG)); TAG2 -- orange --> ATA3((ATA)); ATA3 -- orange --> TAT2((TAT)); TAT2 -- orange --> TGG2((TGG)); TGG2 -- orange --> AAT2((AAT));
```

Figure 5.5: Non-compacted DBG: Example of the INTRA tab with a comparison DBG between two sequences. The first sequence is shown in red, the second sequence is shown in green.

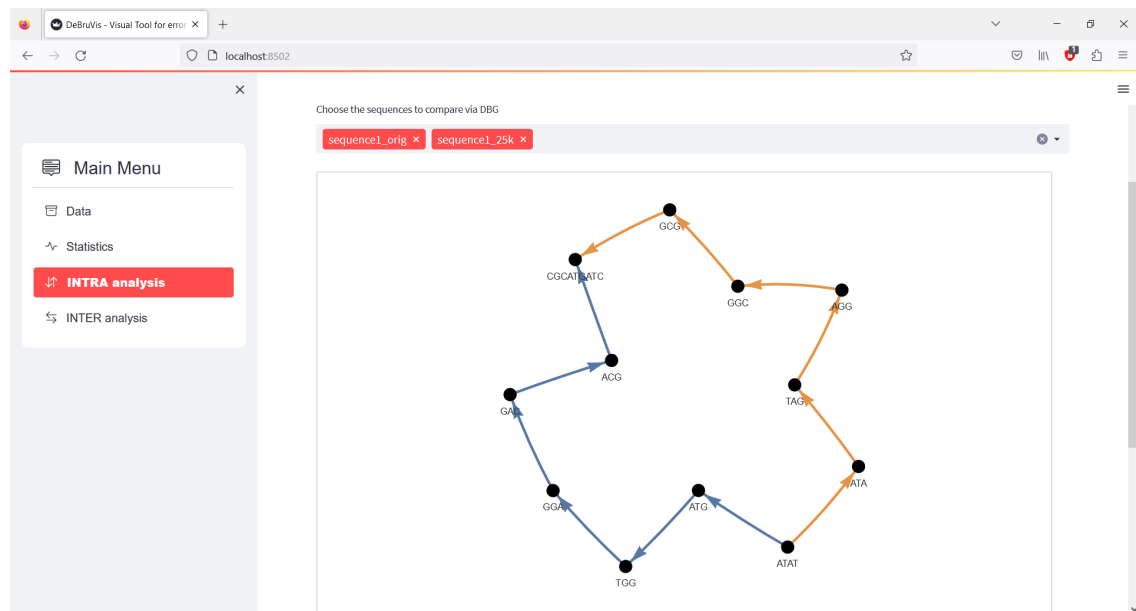


Figure 5.6: Compacted DBG: Example of the INTRA tab with a comparison DBG between two sequences. All nodes of subsequences that are common with all sequences in the set have been merged into a single node. The first sequence is shown in blue, the second sequence in orange.

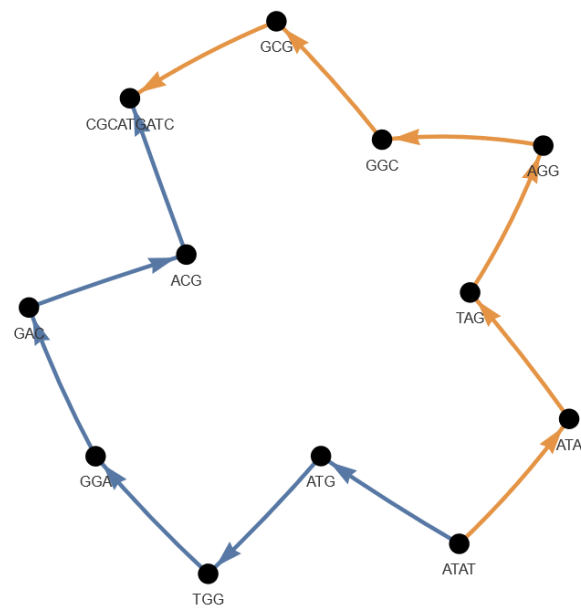


Figure 5.7: Detailed view of the previous figure 5.6. The first sequence is shown in blue, the second sequence in orange. The kmers that were common in 5.5 have been compacted and now have their individual nodes.

5.3 Usecase 2: INTER Analysis with Differently Encoded Sequences

As a second use case, we want to compare how two differently encoded sequences perform in storage by using the INTER tab. As an example for differently encoded sequences, we use the dornröschen dataset and encode it with fountain code and grass code, respectively. Then, we use MESA to simulate both encoded sequences by 1500 months, 3000 months, and 6000 months. We had to choose these relatively short aging times due to performance restrictions of MESA. Afterward, we read all eight sequences into a single FASTA file as presented in 4.1.2 into DeBruVis. Additionally, due to performance restrictions of our tool, which we will detail in 5.4, we reduce the sequences to a length of 1000. In the Data tab, we check the box for INTER analysis this time and choose the four sequences in fountain encoding as the first set and the four sequences in grass encoding as the second set. Since we have already shown the Statistics tab in 5.2, we skip this step and proceed into the INTER analysis.

There, we build the first De Bruijn Graph from all four time points of the fountain encoded data set. We then build the second De Bruijn Graph from all four sequences in grass encoding. The result is shown in figure 5.8.

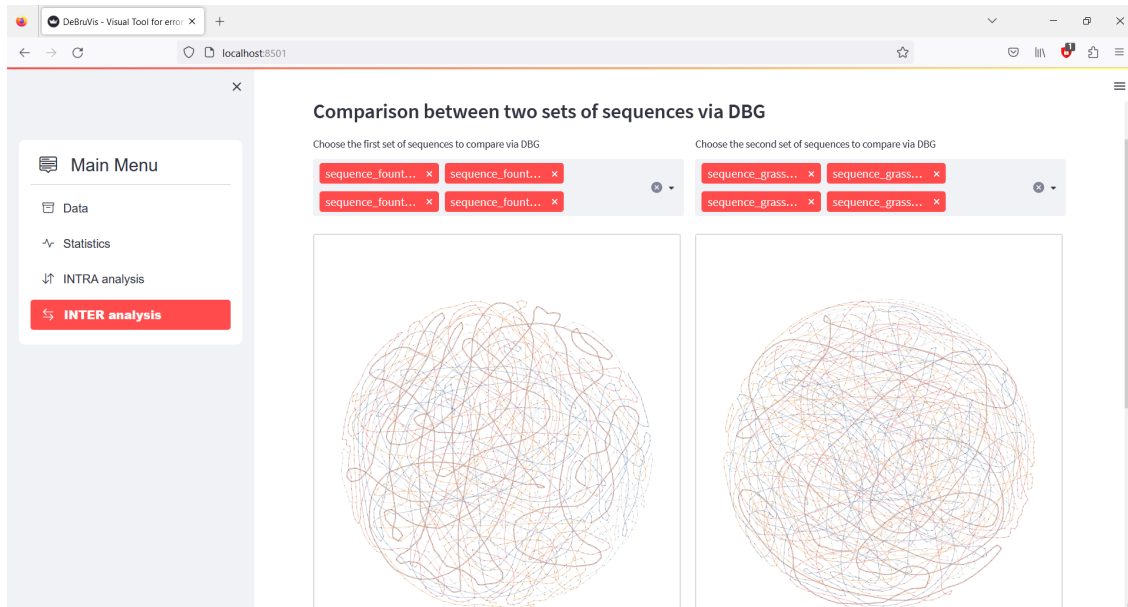


Figure 5.8: Example of the INTER tab, in which we can compare two sets of sequences to each other.

5.4 Performance Analysis

Finally, we want to explore the performance of the tool to give users an understanding of what sequence lengths and quantities it is still possible to work with. We compared different sequence lengths and sequence counts and determined the runtime of each main function in the pipeline to see which one takes the most time. To test the limits of the tool we chose to run the pipeline with 10, 100, and 1000 sequences and sequence lengths and set the kmer size to 3.

The following performance tests were performed on a laptop with Windows 10 Home (Version 22H2) 64bit, an AMD Ryzen 5 3500U processor with Radeon Vega Mobile Gfx and 2.10 GHz, no dedicated graphics card, and 8 GB RAM (5.92 GB usable).

The first tests, as shown in table 5.1, table 5.2, and table 5.3, leave the sequence length at 10 and increase the sequence number. Since a comparison between more than ten sequences will already meet the limits of a colored graph visualization, we expect these cases as less representative of realistic user inputs. Nonetheless, the limits of DeBruVis should be tested.

sequence length	sequence number	k	operation	runtime in ms
10	10	3	Comparing the sequences	144.011
10	10	3	Combining the streak lists	0.000
10	10	3	Preparing for DBG construction	51.003
10	10	3	Optional: Compacting the sequences	162.013
10	10	3	Building the graph	32.023
10	10	3	Build Visualization from graph	36.995
10	10	3	Render graph	2.986

Table 5.1: Testing the runtime of the entire pipeline with a sequence length of 10 and 10 sequences in the set.

The preceding results show that most functions increase their runtime by roughly the same factor as the sequence number. An exception to this is building the visualization from the graph, which has a much steeper increase. Notably, even running the pipeline with a sequence number of 1000 can be performed in a few seconds.

Next, we leave the sequence length at 100 and increase the sequence number again. With respect to realistic user input, a sequence length of 100 is still very probable; however, the sequence number becomes less realistic the more it increases. The following tables 5.4, 5.5, and 5.6 show the results of these runtime tests.

Apart from the runtimes being higher overall due to increased sequence length, we can see almost identical trends to the previous tests. A clear outlier this time is building the visualization from the graph, which increases by a factor of 100 from table 5.5 to table 5.6. Especially the runtime for the last test with a sequence

sequence length	sequence number	k	operation	runtime in ms
10	100	3	Comparing the sequences	997.099
10	100	3	Combining the streak lists	0.000
10	100	3	Preparing for DBG construction	239.020
10	100	3	Optional: Compacting the sequences	1458.107
10	100	3	Building the graph	480.014
10	100	3	Build Visualization from graph	97.01
10	100	3	Render graph	14.001

Table 5.2: Testing the runtime of the entire pipeline with a sequence length of 10 and 100 sequences in the set.

number of 1000 now takes more than seven minutes instead of a couple of seconds.

In the final test run, we leave the sequence length at 1000 and increase the sequence number. In terms of realistic user input, especially the first test of this run could realistically appear. We excluded the test with a sequence length of 1000 and a sequence number of 1000 since it took too long to be feasible in any way. Tables 5.7 and 5.8 show the remaining results for this run.

These results show a less steep increase of about a factor of 5 in most pipeline procedures than before. However, building the visualization from the graph has a runtime increase of more than 50 times from when increasing the number of sequences from 10 to 100. Most importantly, the more realistic case with a sequence number of 10 still runs in about 24 seconds.

We could see roughly a linear increase in most functions within all these test runs. At the same time, the procedure of building the interactive visualization from `pyvis.network` had a much steeper increase in runtime.

sequence length	sequence number	k	operation	runtime in ms
10	1000	3	Comparing the sequences	10142.825
10	1000	3	Combining the streak lists	2.003
10	1000	3	Preparing for DBG construction	2903.245
10	1000	3	Optional: Compacting the sequences	14600.937
10	1000	3	Building the graph	3580.248
10	1000	3	Build Visualization from graph	1844.163
10	1000	3	Render graph	70.999

Table 5.3: Testing the runtime of the entire pipeline with a sequence length of 10 and 1000 sequences in the set.

Lastly, figure 5.9 will provide an overview of the runtime of all parts of the pipeline.

We conclude that DeBruVis is an interactive visualization tool that attempts to leverage compaction and coloring of De Bruijn Graphs to visualize errors in DNA-stored data, but struggles in both visual and computational performance when used with long sequences or sets of a high number of sequences.

The github repository for this thesis, including the source code, use case data and results can be found under <https://github.com/FloSchwa/MastersThesisDeBruVis>.

sequence length	sequence number	k	operation	runtime in ms
100	10	3	Comparing the sequences	339.026
100	10	3	Combining the streak lists	0.000
100	10	3	Preparing for DBG construction	234.04
100	10	3	Optional: Compacting the sequences	485.017
100	10	3	Building the graph	331.022
100	10	3	Build Visualization from graph	77.005
100	10	3	Render graph	11.001

Table 5.4: Testing the runtime of the entire pipeline with a sequence length of 100 and 10 sequences in the set.

sequence length	sequence number	k	operation	runtime in ms
100	100	3	Comparing the sequences	3224.267
100	100	3	Combining the streak lists	0.000
100	100	3	Preparing for DBG construction	1917.146
100	100	3	Optional: Compacting the sequences	4209.341
100	100	3	Building the graph	4033.196
100	100	3	Build Visualization from graph	3033.245
100	100	3	Render graph	67.990

Table 5.5: Testing the runtime of the entire pipeline with a sequence length of 100 and 100 sequences in the set.

sequence length	sequence number	k	operation	runtime in ms
100	1000	3	Comparing the sequences	28078.256
100	1000	3	Combining the streak lists	0.989
100	1000	3	Preparing for DBG construction	18012.642
100	1000	3	Optional: Compacting the sequences	56055.710
100	1000	3	Building the graph	27970.161
100	1000	3	Build Visualization from graph	320297.603
100	1000	3	Render graph	1224.093

Table 5.6: Testing the runtime of the entire pipeline with a sequence length of 100 and 1000 sequences in the set.

sequence length	sequence number	k	operation	runtime in ms
1000	10	3	Comparing the sequences	3494.267
1000	10	3	Combining the streak lists	0.000
1000	10	3	Preparing for DBG construction	2853.215
1000	10	3	Optional: Compacting the sequences	4623.347
1000	10	3	Building the graph	4785.362
1000	10	3	Build Visualization from graph	7567.570
1000	10	3	Render graph	129.011

Table 5.7: Testing the runtime of the entire pipeline with a sequence length of 1000 and 10 sequences in the set.

sequence length	sequence number	k	operation	runtime in ms
1000	100	3	Comparing the sequences	28994.710
1000	100	3	Combining the streak lists	0.997
1000	100	3	Preparing for DBG con- struction	13056.45
1000	100	3	Optional: Compacting the sequences	24156.288
1000	100	3	Building the graph	22440.831
1000	100	3	Build Visual- ization from graph	422226.118
1000	100	3	Render graph	783.220

Table 5.8: Testing the runtime of the entire pipeline with a sequence length of 1000 and 100 sequences in the set.

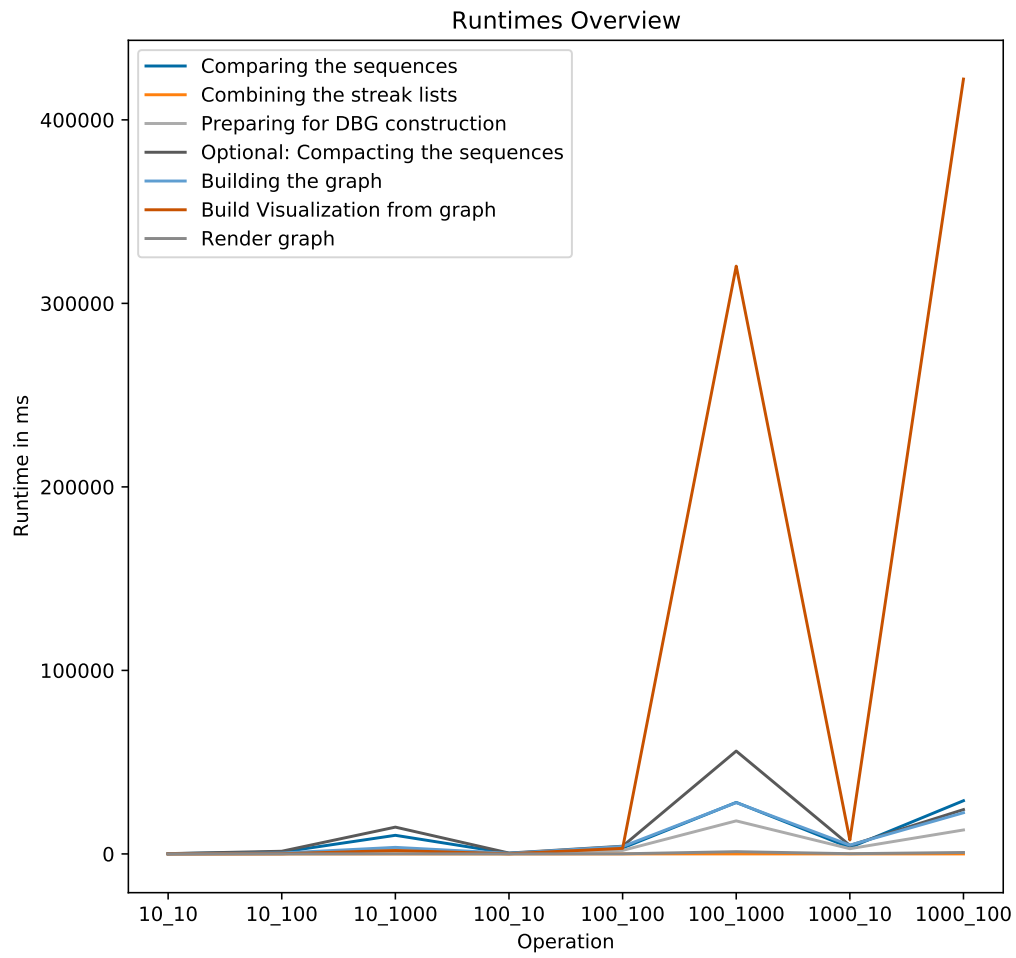


Figure 5.9: An overview of the runtime of the different steps of the pipeline.

6. Discussion

As presented in chapter 5, we provided an interactive visualization tool that builds on an ‘overview first - details second’ approach to give users an understanding of the evolution of DNA-stored data over time. The multi-page application style allows users to traverse the process of understanding and later exploring the data step by step. Furthermore, by only showing necessary components and hiding unused ones through check-boxes, we minimize the information on the screen at any time. This approach should further help readability within the tool. The workflow with the tool is generally structured from top to bottom to support intuitive user behavior and guide them on their journey of understanding and exploring their data.

The different tabs within the tool have their distinct purposes and build on each other. The Statistics tab shows a basic overview that should allow for a first understanding of the errors within the sequences. However, the graphs could be changed, and new components could be added to enhance this first overview. Afterward, the INTRA and INTER analysis tabs allow users to visually and interactively explore the sequences and compare them to each other. A possible issue with these tabs is that users might not know what to do if they do not know the background behind De Bruijn Graphs as a comparison structure for DNA sequences. Additional hints could be given to them via text boxes or tooltips.

Another possible issue of our work is the acquisition of the data. To compare the data at different times during storage, the data has to be retrieved from time to time. It is questionable if the cost of retrieving the data is worth being able to explore the changes that happened to it. Additionally, since only a single FASTA file can be read into the tool, preprocessing must be conducted on the data.

Overall, we are not sure whether colored graphs are an optimal way to visualize the comparison between sequences since their possible length and the number of compared sequences exert a toll on the readability of the graph. Additionally, identifying the errors only by branching might not sufficiently help users identify them in the graph visualization. Further visual cues could be added to improve this,

such as marking the base error in the node label. The current kmer size set to three is not very relevant for synthetic data storage but can be seen as a starting point. The functions provided in the back-end of this thesis can be run with varying kmer counts. Setting the kmer count to more than three will reduce the number of nodes and by that, the size of the graphs.

Finally, the performance of the tool is practical enough to support the most realistic user inputs but meets its limits rather quickly in regard to the possible length of DNA sequences. The biggest factor holding the tool back in terms of performance is the application of the interactive graph visualization from `pyvis.network`. Currently, the INTER analysis only supports the comparison of two De Bruijn Graphs. If this number is increased in the future to compare more graphs, it will have an additional toll on the performance.

7. Conclusion

The primary goal of this thesis was to implement an interactive visualization tool that is capable of examining sequences and the changes that happen to them during storage and visualizing errors. A secondary goal was to show that De Bruijn Graphs are useful for this visualization task.

Over the course of this thesis, we provided an interactive visualization tool called DeBruVis. This tool can, given a data set with DNA sequences, give an overview of the errors within these sequences and also visualize them by using De Bruijn Graphs. We enhanced these De Bruijn Graphs with coloring and compaction to help distinguish sequences within the graph and decrease the visual space of the graph.

Following this thesis, we expect researchers in the domain of DNA data storage to try this tool and use it to visually explore how their data evolved over time. However, the current version of DeBruVis has to be seen as a minimal working example, and, as mentioned in chapter 6, there is still room for improvement of it in many aspects. Furthermore, DeBruVis' performance is lacking with increasing sequence length and number. However, as mentioned in 6, these numbers are less realistic in terms of user input.

For future research, both the back-end and the front-end of DeBruVis could be revisited. In the back-end, a more efficient alternative to the `pyvis.network` functions could be put in place, in case one exists. In the front-end, additional charts and graphs could be added to enhance the overview inside the Statistics tab. Additionally, the handling and information of the INTRA and INTER tab could be made more intuitive to guide users through the exploration of their sequences. In the same vein, the compaction of the graphs turned out to be less helpful than expected and could be neglected.

Glossary

DNA data storage: Storing data (images, text, ...) in artificial DNA by encoding it using an encoding scheme like Huffman Code. This data can then be stored and later decoded.

De Bruijn Graphs (DBG'S): A graph structure invented by Nicholaas De Bruijn. Mainly used for genome assembly up until now, but also has merit for DNA data storage.

DNA bases: DNA sequences consists of the amino acids A, C, G, or C (short of Adenine, Cytosin, Guanin, and Thymin).

k-mer: A k-long sequence of bases.

DNA base errors: During most stages in DNA data storage and synthesis, errors can occur. These errors are divided into three types. Substitution, Deletion and Insertion of a base.

INTRA analysis: We call the comparison of single sequences from a set of sequences INTRA analysis. This can for example be a single sequence at different timepoints.

INTER analysis: We call the comparison of two sets of sequences INTER analysis. This can for example be the entire stored dataset with different encodings.

Compact De Bruijn Graphs: A representation of the De Bruijn Graphs, in which unitigs (non-branching subsequences) are compacted.

Colored De Bruijn Graphs: A representation of the De Bruijn Graph, in which

each edge is assigned a color based on the sample it comes from.

MESA: A tool that can be used to simulate aging of a sequence of a set of sequences.

kmer-list / kmer-df: A list of all kmers belonging to a sequence, ordered by their index in the sequence and assigned an identifier for the sample they originated from.

conflict-dataframe: A supporting structure that, given two sequences, outputs the indices at which their kmers are equal.

DeBruVis: The interactive visualization tool originating from this thesis with the goal to help users identify errors and show the usefulness of DBG's in DNA data visualization.

streamlit: A python package for building web applications, used to implement the front-end of this thesis.

networkx: A python package for building graphs from list of nodes and edges, used to implement the De Bruijn Graphs in this thesis.

pyvis.network: A python package for building interactive visualizations from graphs, used to visualize the De Bruijn Graphs of this thesis.

References

- [1] Yiming Dong et al. “DNA storage: research landscape and future prospects”. In: *National Science Review* 7.6 (2020), pp. 1092–1107.
- [2] Chisom Ezekannagha et al. “Design considerations for advancing data storage with synthetic DNA for long-term archiving”. In: *Materials Today Bio* 15 (2022), p. 100306.
- [3] Siddhant Shrivastava and Rohan Badlani. “Data storage in DNA”. In: *International Journal of Electrical Energy* 2.2 (2014), pp. 119–124.
- [4] Catherine Taylor Clelland, Viviana Risca, and Carter Bancroft. “Hiding messages in DNA microdots”. In: *Nature* 399.6736 (1999), pp. 533–534.
- [5] Carter Bancroft et al. “Long-term storage of information in DNA”. In: *Science* 293.5536 (2001), pp. 1763–1765.
- [6] Marvin H Caruthers. “The chemical synthesis of DNA/RNA: our gift to science”. In: *Journal of biological chemistry* 288.2 (2013), pp. 1420–1427.
- [7] SL Beaucage and Marvin H Caruthers. “Deoxynucleoside phosphoramidites—a new class of key intermediates for deoxypolynucleotide synthesis”. In: *Tetrahedron letters* 22.20 (1981), pp. 1859–1862.
- [8] Anu Stella Mathews, Haikang Yang, and Carlo Montemagno. “Photo-cleavable nucleotides for primer free enzyme mediated DNA synthesis”. In: *Organic & Biomolecular Chemistry* 14.35 (2016), pp. 8278–8288.
- [9] Edward A Motea and Anthony J Berdis. “Terminal deoxynucleotidyl transferase: the story of a misguided DNA polymerase”. In: *Biochimica et Biophysica Acta (BBA)-Proteins and Proteomics* 1804.5 (2010), pp. 1151–1166.
- [10] Weida D Chen et al. “Combining data longevity with high storage capacity—layer-by-layer DNA encapsulated in magnetic nanoparticles”. In: *Advanced Functional Materials* 29.28 (2019), p. 1901672.

- [11] Thomas J Anchordoquy and Marion C Molina. “Preservation of DNA”. In: *Cell Preservation Technology* 5.4 (2007), pp. 180–188.
- [12] James Bornholt et al. “A DNA-based archival storage system”. In: *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*. 2016, pp. 637–649.
- [13] Nicolaas Govert De Bruijn. “A combinatorial problem”. In: *Proceedings of the Section of Sciences of the Koninklijke Nederlandse Akademie van Wetenschappen te Amsterdam* 49.7 (1946), pp. 758–764.
- [14] David A Huffman. “A method for the construction of minimum-redundancy codes”. In: *Proceedings of the IRE* 40.9 (1952), pp. 1098–1101.
- [15] Roy D Sleator. *Digital biology: a new era has begun*. 2012.
- [16] Nick Goldman et al. “Towards practical, high-capacity, low-maintenance information storage in synthesized DNA”. In: *nature* 494.7435 (2013), pp. 77–80.
- [17] Reinhard Heckel, Gediminas Mikutis, and Robert N Grass. “A characterization of the DNA data storage channel”. In: *Scientific reports* 9.1 (2019), p. 9663.
- [18] Chengtao Xu et al. “Uncertainties in synthetic DNA-based data storage”. In: *Nucleic acids research* 49.10 (2021), pp. 5451–5469.
- [19] Lifu Song et al. “Robust data storage in DNA by de Bruijn graph-based de novo strand assembly”. In: *Nature Communications* 13.1 (2022), p. 5361.
- [20] Han Mao Kiah, Gregory J Puleo, and Olgica Milenkovic. “Codes for DNA sequence profiles”. In: *IEEE Transactions on Information Theory* 62.6 (2016), pp. 3125–3146.
- [21] Rayan Chikhi, Antoine Limasset, and Paul Medvedev. “Compacting de Bruijn graphs from sequencing data quickly and in low memory”. In: *Bioinformatics* 32.12 (2016), pp. i201–i208.
- [22] Einar Andreas Rødland. “Compact representation of k-mer de Bruijn graphs for genome read assembly”. In: *BMC bioinformatics* 14.1 (2013), pp. 1–19.
- [23] Zamin Iqbal et al. “De novo assembly and genotyping of variants using colored de Bruijn graphs”. In: *Nature genetics* 44.2 (2012), pp. 226–232.
- [24] Martin D Muggli et al. “Succinct colored de Bruijn graphs”. In: *Bioinformatics* 33.20 (2017), pp. 3181–3187.

- [25] Qi Wang et al. “3DScapeCS: application of three dimensional, parallel, dynamic network visualization in Cytoscape”. In: *BMC bioinformatics* 14 (2013), pp. 1–8.
- [26] Robert Kincaid, Allan Kuchinsky, and Michael Creech. “VistaClara: an expression browser plug-in for Cytoscape”. In: *Bioinformatics* 24.18 (2008), pp. 2112–2114.
- [27] Michael Schwarz et al. “MESA: automated assessment of synthetic DNA fragments and simulation of DNA synthesis, storage, sequencing and PCR errors”. In: *Bioinformatics* 36.11 (2020), pp. 3322–3326.
- [28] Yaniv Erlich and Dina Zielinski. “DNA Fountain enables a robust and efficient storage architecture”. In: *science* 355.6328 (2017), pp. 950–954.
- [29] Robert N Grass et al. “Robust chemical preservation of digital information on DNA in silica with error-correcting codes”. In: *Angewandte Chemie International Edition* 54.8 (2015), pp. 2552–2555.
- [30] Wes McKinney et al. “pandas: a foundational Python library for data analysis and statistics”. In: *Python for high performance and scientific computing* 14.9 (2011), pp. 1–9.
- [31] Aric Hagberg and Drew Conway. “Networkx: Network analysis with python”. In: URL: <https://networkx.github.io> (2020).
- [32] Giancarlo Perrone, Jose Unpingco, and Haw-minn Lu. “Network visualizations with Pyvis and VisJS”. In: *arXiv preprint arXiv:2006.04951* (2020).
- [33] Vladimir I Levenshtein et al. “Binary codes capable of correcting deletions, insertions, and reversals”. In: *Soviet physics doklady*. Vol. 10. 8. Soviet Union. 1966, pp. 707–710.
- [34] John D Hunter. “Matplotlib: A 2D graphics environment”. In: *Computing in science & engineering* 9.03 (2007), pp. 90–95.